

category: avalonia

tags: [ui-lag, sql-timeout, dispatcher, async-load, kiosk-config]

severity: high

created: 2026-07-23

updated: 2026-07-23

project-origin: iPGSv4 (IPGS.Kiosk.Avalonia, KIOSK VERTICAL)

Màn hình Cấu hình (Window) treo UI thread nhiều giây khi SQL/parking chưa kết nối vì đọc DB đồng bộ trong constructor

Tình huống gặp phải

User báo: "khi không có kết nối SQL, parking hoặc chưa có config thì khi bấm vào màn config sẽ rất lag".

Project `IPGS.Kiosk.Avalonia` — màn `ConnectionConfigWindow` (Avalonia port của `frmConnectionConfig WinForms`), mở từ `MainView.OnSettingClicked` bằng `new ConnectionConfigWindow()` rồi `await ShowDialog(owner)`.

Triệu chứng / Lỗi

Bấm icon Cài đặt → nhập mật khẩu đúng → toàn bộ UI kiosk (kể cả overlay dialog) đứng hình vài giây đến ~30 giây trước khi màn Cấu hình hiện ra, đặc biệt rõ khi máy chưa từng cấu hình SQL Server hoặc server không reachable.

Nguyên nhân gốc rễ (Root Cause)

`ConnectionConfigWindow` constructor gọi `_vm.Load()` đồng bộ, trên UI thread, trước khi `ShowDialog`. `ConnectionConfigViewModel.Load()` gọi `KioskServices.Db.FillData("SELECT ... FROM tblKioskConfig")` để đọc timer. `KioskDbService.OpenInternal()` (port từ `MDB.cs`) không set `Connect Timeout` trong connection string (mặc định `SqlClient` ~15s) và có logic retry tối đa 2 lần kèm `Thread.Sleep(1000)` giữa các lần — khi SQL chưa cấu hình/không kết nối được, tổng thời gian block UI thread có thể lên tới ~30s ($2 \times 15s + 1s \text{ sleep}$) TRƯỚC KHI cửa sổ kịp hiển thị.

`Save()` cũng có cùng pattern: `ExecuteCommand` ghi timer vào DB chạy đồng bộ trước khi `CloseRequested` — bấm Lưu cũng bị treo tương tự.

Giải pháp

```
// KioskDbService.OpenInternal(): thêm Connect Timeout=3 vào connection string
// → chặn worst-case mỗi lần thử kết nối ở 3s thay vì mặc định ~15s (2 lần retry ⇒ ~7s
// thay vì ~30s)

// ConnectionConfigViewModel: tách Load() thành 2 phần
public void Load() { /* chỉ đọc JSON local - nhanh, không đụng DB */ }
public async Task LoadTimerFromDbAsync()
{
```

```
var dt = await Task.Run(() => KioskServices.Db.FillData("SELECT ..."));
await Dispatcher.UIThread.InvokeAsync(() => { NormalWaitingTimeText = ...;
PaymentWaitingTimeText = ...; });
}

// ConnectionConfigWindow constructor: gọi Load() đồng bộ (nhANH), rồi fire-and-forget
_ = LoadTimerAsync(); // await _vm.LoadTimerFromDbAsync() sau khi form đã hiển thị

// Save(): đóng form (Confirmed=true + CloseRequested) TRƯỚC, ghi DB timer trong
Task.Run sau
// (fire-and-forget) - Save không còn chờ SQL để đóng dialog.
```

1. Bounded timeout ở tầng connection string (defense-in-depth, không phụ thuộc code gọi có nhớ tách async hay không).
2. Tách phần đọc/ghi DB không quan trọng cho việc hiển thị/đóng form ra khỏi luồng đồng bộ chặn UI.
3. Marshal property update về UI thread qua `Dispatcher.UIThread.InvokeAsync` vì CommunityToolkit `ObservableObject` không tự làm việc này khi set property từ background thread.

Áp dụng lại (How to reuse)

- Khi thấy 1 Window/Dialog Avalonia gọi `_vm.Load()` (hoặc bất kỳ method đồng bộ nào) TRONG CONSTRUCTOR mà bên trong có gọi DB/HTTP/IO chậm → nghi ngờ ngay UI-thread-blocking, đặc biệt nếu method đó có try/catch nuốt lỗi kết nối (dấu hiệu tác giả đã biết resource có thể unreachable nhưng chưa nghĩ tới thời gian chờ).
- Kiểm tra `IKioskDb`/tương đương: connection string có set `Connect Timeout` tương minh chưa? Nếu không → mặc định driver (SqlClient ~15s, có thể lâu hơn tùy platform) nhân với số lần retry = worst-case thực tế.
- Pattern chuẩn cho Window Avalonia có I/O chậm khi mở: constructor chỉ load phần local/nhanh để hiển thị ngay, phần I/O chậm chạy `Task.Run` + cập nhật UI qua `Dispatcher.UIThread` sau khi form đã visible.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `KioskDbService.FillData/ExecuteCommand` là API đồng bộ (chưa có bản `*Async`) — bọc bằng `Task.Run` ở call-site, KHÔNG tự thêm `async` giả vào `KioskDbService` (API đó vẫn block bên trong `SqlConnection.Open()/SqlDataAdapter.Fill()` đồng bộ, chỉ đổi thread thực thi).
- ⚠ Khi update `[ObservableProperty]` từ background thread mà không qua `Dispatcher.UIThread`, binding vẫn có thể "work" trong nhiều trường hợp (tùy Avalonia version) nhưng không đảm bảo — luôn marshal về UI thread cho chắc chắn, đặc biệt với property bind 2 chiều vào `TextBox`.
- ⚠ Đóng form trước khi ghi xong DB (Save) nghĩa là nếu ghi timer thất bại, user sẽ không biết — chấp nhận được vì đây là timer phụ trợ (auto_return_main/wait_for_payment), không phải dữ liệu giao dịch; KHÔNG áp dụng pattern "đóng trước, ghi sau" cho các luồng ghi dữ liệu quan trọng (payment, transaction) mà không có cơ chế retry/notify riêng.

Tham chiếu

- File: `IPGS.Kiosk.Avalonia/Services/Data/KioskDbService.cs` (OpenInternal),
`Views/ConnectionConfig/ConnectionConfigViewModel.cs`,
`Views/ConnectionConfig/ConnectionConfigWindow.axaml.cs`
- GOTCHAS entry: G-configscreen-lag-no-sql
- Project liên quan: iPGSv4 (IPGS.Kiosk.Avalonia)